Guía paso a paso: mejoras al modelo de puestos de hidratación en AnyLogic

Oct 13, 2026 · @Ramiro Ceballos

1. Punto de partida: merge y rama de trabajo

Todo el trabajo se hizo en la rama `r.ceballos/mejoraComportamientoSimulacion`, creada sobre `valeria-arreglosGral` después de unir con ella la rama del menú. Nada se subió a GitHub: las ramas de Valeria no se tocan en el remoto.

1. Traer las ramas del remoto: `git fetch --all --prune`.
2. Comparar la rama del menú con la de Valeria: `git diff origin/r.ceballos/decisionHidratacion origin/valeria-arreglosGral`. Resultado: `valeria-arreglosGral` ya contenía el menú por Areas y la decisión de hidratación, y agregaba el agente `Puesto` (stock por puesto) y la vista `PuestoDetalle3D`.
3. Crear la rama local de Valeria y unir la del menú: `git checkout -b valeria-arreglosGral --track origin/valeria-arreglosGral` y después `git merge --no-ff --no-commit origin/r.ceballos/decisionHidratacion`.
4. Resolver los conflictos quedándose con la versión de Valeria (`git checkout --ours -- <archivo>`). Ojo con `Experiments.xml`: el merge automático duplicaba parámetros, así que también se restaura la versión de Valeria.
5. Crear la rama de trabajo: `git checkout -b r.ceballos/mejoraComportamientoSimulacion`.

El modelo está en formato *split* de AnyLogic: `PuestosDeHidratacionDeUnaMaraton.alpx` más la carpeta `_alp/`, con un XML por agente (`AOC.*.xml`, `Variables.xml`, `EmbeddedObjects.xml`, `Connectors.xml`, `Levels/Level.level.xml`) y el código de funciones en `Code/*.java`. Todos los cambios de esta guía se pueden hacer desde la interfaz de AnyLogic; la guía indica dónde va cada cosa.

2. Cuatro categorías de corredor

Cada corredor sortea al largar una de cuatro categorías, y esa categoría fija su velocidad, cuánto se deshidrata, cuánto tarda en el puesto y cuánto se fatiga. Se hace con un solo tipo de agente (`Corredor`) parametrizado, que es lo habitual en AnyLogic.

Categoría	Reparto	Cadencia (pasos/min)	Zancada (m)	Ritmo aprox.	Color en el mapa
Elite	3 %	185	1,72	3:05 min/km	dorado
Avanzado	17 %	178	1,32	4:15 min/km	rojo
Intermedio	50 %	170	1,12	5:15 min/km	violeta
Principiante	30 %	160	0,95	6:40 min/km	verde

1. En el agente `Corredor`, crear variables (paleta Agente → Variable): `categoria`, `cadencia`, `zancada`, `velocidadBaseKmh`, `necesidadHidratacion`, `factorTiempoPuesto`, `resistenciaFatiga`, `nivelHidratacion`, `nivelEnergia`, `tiempoHidratacion` y `propensionHidratacion`. Las constantes `ELITE`, `AVANZADO`, `INTERMEDIO`, `PRINCIPIANTE` y `NOMBRES_CATEGORIA` se marcan como *Static* y *Constant*.
2. Crear la función `inicializar(int cat)`. Con un `switch` sortea cadencia y zancada con `normal(sigma, media)` (en AnyLogic el desvío va primero) y calcula `velocidadBaseKmh = cadencia × zancada × 60 / 1000`.
3. En Main, crear `sortearCategoria()` con el reparto de la tabla, y en el bloque `sourceCorredores` → *On exit* escribir `agent.inicializar( sortearCategoria() );`.
4. Crear `velocidadKmh(nivelCalor)`, que aplica sobre la velocidad base el calor, la deshidratación (por debajo del 70 %), "el muro" de energía (por debajo del 35 %) y la fatiga después del km 30.
5. En cada bloque `moveToN` poner *Speed* = `((Corredor) agent).velocidadKmh( nivelCalor )` en km/h, y en *On exit* `((Corredor) agent).avanzarHasta( puesto.get( i ).km, nivelCalor );`, que descuenta hidratación y energía por los km corridos.
6. Agregar el bloque `moveToLlegada` (destino `llegada`) antes del `sinkCorredores`, que antes terminaba en el km 40. En el *On enter* del sink, `registrarLlegada( (Corredor) agent )` guarda el tiempo final por categoría.
7. En el óvalo de la presentación de `Corredor`, *Fill color* dinámico = `colorCategoria()`.

3. Puestos por insumo con stock separado

Qué ofrece cada puesto sale del shapefile real `puestos_hidratacion3.dbf` (columnas `agua`, `isotonica`, `fruta`, `km`). Los geles no están en el archivo y se agregaron en los km 22,6 y 30. Cada insumo de cada puesto tiene su propio stock, capacidad, punto de reposición y contador de faltantes.

1. Leer el `.dbf` y cruzar cada punto con los `GISPoint` `puesto1..puesto15` del mapa por cercanía de coordenadas. Así se obtuvo la oferta de cada puesto, por ejemplo P7 = agua + fruta y P12 = agua + isotónica + fruta.

2. En `Puesto`, crear las constantes estáticas `AGUA = 0`, `ISOTONICA = 1`, `FRUTA = 2` y `GEL = 3`, y los arreglos de 4 posiciones `ofrece`, `stock`, `capacidad`, `puntoReposicion`, `consumidos` y `faltantes`.
3. La población `puesto` se cargaba de una tabla de base de datos (`puestos_maraton`) que no estaba en el repositorio. Se cambió a *Population is: initially contains 15 agents* y los datos van en la función `configurar()`, llamada desde el *On startup* de `Puesto`.
4. `configurar()` carga km, oferta, capacidades (parámetros `capacidadAgua`, `capacidadIsotonica`, `capacidadFruta` y `capacidadGel` de Main) y ubica el agente con `jumpTo( main.puntoPuesto( i ) )`.
5. Crear `atender(Corredor c)`. Elite y avanzados prefieren isotónica; si se acabó lo que prefiere, toma la otra bebida. Toma gel si le falta energía y fruta sobre todo los más lentos. `consumir(int insumo)` descuenta una unidad o cuenta un faltante.
6. En cada `delayHidratacionN` → *On at exit* escribir `puesto.get( i ).atender( (Corredor) agent );`.
7. Corrección de un error previo: los 15 `seizeVoluntariosN` tomaban todos `poolVoluntarios`, el del puesto 1. Cada uno tiene que tomar su `poolVoluntariosN`.
8. Presentación del puesto en el mapa: un óvalo con *Fill color* = `colorTipo()` y *Line color* = `pedidoEnCurso ? red : black`.

4. Entrega en mano, voluntarios en 3D y corredores no atendidos

Los voluntarios esperan en el cordón con la botella en la mano, y el corredor la agarra sin ir hasta la mesa. Los que no pueden ser atendidos se cuentan y se marcan en la escena con un globo: rojo si paró y no había bebida, naranja si no paró porque no había voluntario libre.

1. Lógica de colas: en Main, el parámetro `entregaEnMano` (verdadero) reduce a la mitad el tiempo de atención en `tiempoAtencion(c)`. Cada `poolVoluntariosN` (*ResourcePool*) tiene *Capacity* = `voluntariosPorPuesto`, y `cambiarVoluntarios(n, cantidad)` la cambia en la corrida con `set_capacity`.
2. Agente `Voluntario` dentro de `PuestoDetalle3D`, en una población de 8. Solo se muestran y trabajan los primeros N, con N = voluntarios del puesto (`activo()`). Al entregar camina a la mesa con `moveTo(x, y)`, toma otra botella (evento `TomarBotella`) y vuelve al cordón.
3. Flujo peatonal de la escena: `pedSource → pedSeHidrata (PedSelectOutput) → pedIrAlVoluntario (PedGoTo a un punto) → pedRecibirBotella (PedWait) → pedGoTo1 → pedSink`. Los que no paran van directo de `pedSeHidrata` a `pedGoTo1`.
4. No atendidos por falta de voluntarios: en `decidirHidratacion`, si el corredor quiere parar pero `hayDemasiadaCola( n )` es verdadero, sigue de largo. Eso pasa cuando no hay voluntarios libres (`ResourcePool.idle() == 0`) y la cola (`Seize.size()`) ya tiene

`colaMaximaPorVoluntario` corredores por voluntario. Se marca `c.motivoNoAtendido = 2`.

5. No atendidos por falta de stock: en `Puesto.atender`, si no se pudo dar ni agua ni isotónica, suma `noAtendidosStock` y marca `c.motivoNoAtendido = 1`.
6. En 3D, el peatón copia ese motivo al crearse. Un óvalo con altura (cilindro) a 2,1 m sobre su cabeza tiene *Visible* = `motivoNoAtendido > 0` y *Fill color* = `motivoNoAtendido == 1 ? red : orange`. Si paró sin bebida, el voluntario no le entrega nada (`terminarEntrega`), y la botella del voluntario se oculta cuando el puesto no tiene bebida (`hayBebida()`).

Una condición de `SelectOutput` o `PedSelectOutput` se evalúa dos veces por agente. Si da distinto entre una y otra, la corrida aborta. Por eso la decisión de cada puesto se sortea una sola vez y se guarda en el corredor.

5. Reposición con camión y operario con zorra

Cuando un insumo baja de su punto de reposición sale un camión del depósito por calles reales. Estaciona en una calle aledaña (el circuito está cortado) y un operario lleva la mercadería con una zorra, en tantos viajes como haga falta para llenar el puesto. Son dos tipos de agente nuevos que se mueven sobre el mapa GIS de Main.

El camión trae la carga y la zorra repone en viajes hasta llenar el puesto



ciclo de reposición · camión y zorra

La zorra repite carga y descarga hasta que el puesto queda lleno; recién ahí se cierra el pedido y el camión se va.

1. Crear los agentes `Camion` y `OperarioZorra` (*New agent type*, ubicados en Main) y sus poblaciones vacías `camiones` y `operarios` (*Population is: initially empty*). Tildar *Agents in the space* para que se dibujen en el mapa.
2. Calcular para cada puesto un punto de estacionamiento a unos 130 m perpendiculares al recorrido (`latEstacionamiento`, `lonEstacionamiento` en `configurar()`). El ruteo GIS lo lleva a la calle real más cercana.
3. `Camion.despachar(Puesto p)`: `jumpTo( main.latDeposito, main.lonDeposito )` y después `moveTo( lat, lon )`, que va por las calles con el ruteo del mapa. En la pestaña *Agent actions* del tipo de agente, *On arrival to target location* decide qué hacer según la variable `fase` (YENDO → baja el operario; VOLVIENDO → `main.remove_camiones( this )`).
4. `OperarioZorra` usa `moveToStraight( lat, lon )` (camina en línea recta). Cada carga y descarga dura 1 minuto con eventos dinámicos (`FinCarga`, `FinDescarga`, creados con `create_FinCarga( 1, MINUTE )`).
5. En cada descarga, `Puesto.recibirCarga( unidades )` reparte la carga entre los insumos según lo que le falta a cada uno. Cuando el puesto queda lleno, `finReposicion()` libera el pedido y el camión vuelve al depósito.
6. Velocidades y cantidades son parámetros de Main: `velocidadCamionKmh` (25), `velocidadZorraKmh` (3,5) y `capacidadZorra` (900 vasos por viaje, unos 105 L).

6. Política de reposición con bibliografía

Cada puesto pide el camión cuando su stock llega a la demanda esperada durante el tiempo de entrega más un stock de seguridad. Es el modelo clásico de revisión continua y reemplaza al 35 % fijo, que dejaba vacíos los primeros puestos antes de que llegara el camión.

```
ROP = d \cdot L + z \sqrt{d \cdot L}
```

- d es la demanda en unidades por minuto: los corredores que pasan por minuto (60 × `tasaLlegada`) por los vasos que necesita cada uno. Los vasos salen de la ingesta (466 mL/h, ajustada por calor) por las horas desde el puesto anterior, dividido 116 mL por vaso. Si la demanda medida en la corrida es mayor, se usa esa.
- L es el tiempo de entrega en minutos: el camión, la caminata de la zorra y 1 minuto de descarga.
- z = 1,65 apunta a que el 95 % de los pedidos llegue antes de que se acabe el stock. Con demanda Poisson, el desvío durante la entrega es la raíz de d × L.

Dato	Valor	Fuente
Ingesta de líquido en carrera	466 ± 279 mL/h	Jiménez-Alfageme et al. 2025, Sports Medicine – Open 11:26, Maratón de Sevilla 2022
Volumen por vaso	116 mL (1.036.003 vasos para 27.222 corredores)	Cheuvront et al. 2019, Med Sci Sports Exerc 51(5):1040-1046 (PubMed 30531484), Maratón de Boston 2017
Rango de referencia	0,55 ± 0,34 L/h en elite	Beis et al. 2012, Clin J Sport Med 22(3)

1. En `Puesto`, crear `demandaPlanificada(k)`, `tiempoEntregaEstimado()` y `actualizarPuntoReposicion(k)`, que aplica la fórmula y topea en el 75 % de la capacidad. Si choca con el tope avisa "capacidad corta".
2. Crear el evento dinámico `MedirDemanda`, que cada minuto mide lo consumido más lo faltante (promedio móvil exponencial), recalcula el punto y se vuelve a programar con `create_MedirDemanda( 1, MINUTE )`.
3. En Main, nuevos parámetros: `consumoLiquidoMlHora`, `volumenVasoMl`, `gelesPorCorredor` (0,5) y `frutaPorCorredor` (0,4), los dos últimos como supuestos sin fuente, más `zNivelServicio`, `topePuntoReposicion` y `factorRecorridoCalles`.
4. Como la unidad pasó a ser un vaso de unos 116 mL, las capacidades se llevaron a una escala realista: agua 3000, isotónica 2500, fruta y gel 1000.

7. Interfaz: navegación, inicio y mapa 2D

Cada pantalla es una *View Area* y una barra superior repetida en todas permite saltar entre ellas. Reemplaza los botones del menú y los "◀ Menú" de cada sección.

1. En Main, definir las áreas (paleta Presentación → *View Area*): `areaMenu`, `areaMapa2D`, `areaLogica` y `areaEjecucion`, cada una del tamaño de su contenido.
2. Crear la función `irA(String seccion)`: guarda la sección en `pantalla` y llama `navigateTo()` sobre el área que corresponde. En *On startup* de Main: `irA( "MENU" );`.
3. Barra superior: un rectángulo oscuro, el título y cinco rectángulos redondeados con texto. Cada uno tiene en *On click* `irA( "<SECCION>" );` y en *Fill color* dinámico `colorTab( "<SECCION>" )`, que pinta de naranja la pestaña activa.

4. Inicio: título, integrantes, cinco indicadores con texto dinámico (tiempo de carrera `formatoHoras( time() )`, corredores en carrera `sourceCorredores.count() - sinkCorredores.count()`, llegados, faltantes y camiones en ruta `camiones.size()`) y cuatro tarjetas que llevan a cada sección.
5. Mapa 2D: un panel de referencias con los colores de corredores y puestos. En el óvalo del puesto, *On click* = `main.verPuesto3D( getIndex() );` abre su vista 3D.

Al correr el modelo, AnyLogic PLE lo muestra en su propio navegador Chromium. Conviene tener un solo visor abierto: con dos, los clics sobre figuras fallan (`XYtoShapeXY`), y si se cierra esa ventana termina la corrida.

8. Escenas 3D por puesto, entorno real y cámaras

Hay una escena 3D por cada uno de los 15 puestos, con una avenida de 140 m decorada según el entorno real del lugar y cuatro cámaras. Solo la escena que se está mirando genera peatones, para no cargar el simulador.

1. Convertir `puestoDetalle3D` en la población `puestosDetalle3D` de 15 agentes (sin presentación en Main, *not in environment*). Cada escena muestra el puesto de su mismo índice con `miPuesto()` = `main.puesto.get( getIndex() )`.
2. Dentro de `PuestoDetalle3D`, crear la View Area `areaPuesto3D` con la barra superior, el selector P1–P15 (`main.verPuesto3D( k )`), la ventana 3D (*Window 3D*) y un panel de estado.
3. En cada `decisionHidratacionN` de Main, en *On exit (true)* y *On exit (false)*: `puestosDetalle3D.get( i ).ingresarCorredor( (Corredor) agent, true/false );`. En la escena, `PedSource.inject( 1 )` crea el peatón con la velocidad del corredor.
4. Calle: el *markup* peatonal (`areaCalle`, `arribalLine`, `exitLine`) se estira a 140 m × 14 m. Encima van rectángulos con texturas nativas: `tarmac` (asfalto), `concrete` (veredas de 15 cm de alto) y `grass`. Además, líneas de carril, árboles, faroles, bancos, público y el cielo (*Environment* = `SKYBOX_1` en el tipo de agente).
5. Entorno real: con las coordenadas de cada puesto se consultó OpenStreetMap (Overpass para calles, edificios y parques en 120 m; Nominatim para el barrio). Con eso se clasificó cada puesto en edificios, parque o mixto y se guardó en `Puesto.configurar()` (`calle`, `barrio`, `entorno`, `cercaDelAgua`). Los edificios y la arboleda tienen *Visible* dinámico según `miPuesto().entorno`.
6. Objetos 3D: se copiaron de la biblioteca de AnyLogic a la carpeta `3d/` del modelo (edificios, faroles, bancos, parada de colectivo, árboles) y se registraron en `ModelResources.xml`.

7. Cámaras: `camera` (desde la mesa), `camaraLlegada`, `camaraAerea` y `camaraCorredor`, esta última dentro de la presentación del peatón, así se mueve con él. `verCamara(c)` llama `window3d.setCamera( camara, true, 1500 )`, que hace una transición suave. En AnyLogic, *Rotation Z* = 0 mira al este y los ángulos positivos giran hacia el sur.

9. Ajustes por puesto y temperatura en °C

Debajo de la ventana 3D, cada puesto tiene una barra de ajustes: stock máximo y mínimo del insumo elegido, demora del camión, voluntarios y la temperatura del día. También muestra en cuántos minutos se agota el stock y cuánto le falta al camión para llegar.

Slider	Rango	Qué hace	Función que llama
Stock máximo	100 a 10.000	Capacidad del insumo elegido	`Puesto.ajustarCapacidad(k, valor)`
Stock mínimo	0 a 10.000	Punto de reposición manual; "Auto" vuelve a la fórmula	`ajustarMinimo(k, valor)` / `minimoAutomatico(k)`
Camión	1 a 120 min	Minutos del depósito al estacionamiento; "Auto" vuelve a la estimación por distancia	`ajustarDemoraCamion(min)` / `demoraAutomatica()`
Voluntarios	1 a 15	Voluntarios del puesto	`main.cambiarVoluntarios(i, n)`
Temperatura	0 a 40 °C	Temperatura de toda la carrera	`main.ajustarTemperatura(t)`

1. Agregar cada slider (paleta Controles → *Slider*) sin *Link to*, con el código de la tabla en *Action*. Dentro de *Action*, `value` es el valor elegido.
2. Crear `sincronizarControles()`, que pone cada slider en el valor actual del puesto con `slider.setValue( valor, false )`. El `false` evita que se dispare la acción. Se llama al entrar a la escena y al cambiar de insumo.
3. Demora fija del camión: en `Camion.despachar`, si el puesto tiene demora manual, se usa `moveToInTime( lat, lon, minutos, MINUTE )`, que ajusta la velocidad para llegar justo en ese tiempo. Cuánto le falta sale de `camionAsignado.timeToArrival( MINUTE )`.
4. Agotamiento: `minutosParaAgotar(k)` = stock / demanda medida por minuto. Si todavía no hay consumo, el panel dice "sin consumo por ahora".

5. Temperatura: `temperaturaC` pasa a ser el parámetro de Main (20 °C por defecto), y `nivelCalor` se calcula a partir de ella:

```
nivelCalor = \min\left(1, \max\left(0, \frac{T - 10}{20}\right)\right)
```

Con 10 °C o menos vale 0 (fresco), con 30 °C o más vale 1 (agobiante), y 20 °C da 0,5, el valor que tenía antes. Ese nivel ya alimentaba la intención de hidratarse (`decidirHidratacion`), la deshidratación por km, la velocidad y la demanda planificada, así que la temperatura afecta todo eso a la vez. El rango 10–30 °C es un supuesto del modelo, en línea con que el rendimiento en maratón empeora a medida que la temperatura sube por encima de unos 10 °C.

10. Pestaña Métricas

Métricas quedó solo con datos de la competencia, en cuatro bloques de arriba hacia abajo. Los sliders se sacaron, porque ahora están en cada vista 3D.

Bloque	Contenido	Cómo se arma
Resultados por categoría	Llegados, tiempo promedio y ritmo promedio (min/km) de Elite, Avanzado, Intermedio y Principiante	Textos con *Text* dinámico: `llegadosPorCategoria[i]`, `tiempoPromedioCategoria(i)` y `ritmoPromedioCategoria(i)`
Resumen	Corredores en carrera, llegados, temperatura, no atendidos sin bebida y sin voluntario, camiones despachados	Rectángulos redondeados con título y valor dinámico
Inventario por puesto	Una fila por puesto: calle, stock / capacidad por insumo, voluntarios, faltantes, no atendidos y estado de la reposición	Una celda por texto. *Color* dinámico `colorCeldaStock(i, k)`: rojo sin stock, naranja por debajo del punto de reposición
Stock en tiempo real	Un gráfico por insumo, una línea por puesto	*Time Plot* (paleta Análisis), dato tipo *Value* = `puesto.get(i).stock[ Puesto.AGUA ]`, cada 1 minuto, ventana de 900 min, eje fijo de 0 a la capacidad

1. Borrar los sliders de voluntarios y los textos largos que había (`textResultadosCategorias`, `textInventarioPuestos`).

2. Armar cada tabla con un rectángulo de fondo, una fila de encabezados en negrita y una fila de textos por elemento. Las filas pares llevan un fondo gris claro para que se lean mejor.
3. Los textos de la tabla de inventario usan funciones de Main que devuelven el texto ya armado (`celdaStock` , `textoReposicionCorto`), así cada celda queda corta y alineada.
4. Todos los elementos tienen *Visible* = `pantalla.equals("EJECUCION")` , para no calcularlos cuando la pantalla no se ve.
5. `areaEjecucion` se agrandó a 1200 × 1300 para que entren las tablas y los gráficos.

11. Personas animadas y vista 3D en tiempo real

En la vista 3D los corredores cruzaban la escena en unos 2 segundos, y las personas eran figuras quietas o un mismo modelo que se deslizaba. Esta sección cambia dos cosas: la escena pasa a correr en tiempo real, y corredores, voluntarios y público usan personas animadas de la biblioteca de AnyLogic.

Por qué iban tan rápido. El modelo mide el tiempo en minutos (*Model time units: minutes*), y AnyLogic toma la escala de tiempo real como "unidades de modelo por segundo real". En x1 pasa un minuto de carrera por segundo: un corredor a 12 km/h recorre 200 m en un segundo de pantalla. Las velocidades del modelo estaban bien; lo que había que cambiar era la escala.

1. En Main, agregar tres variables: `velocidad3D` (double, 1), `escalaFuera3D` (double, 1) y `tiempoRealFuera3D` (boolean, true).
2. Crear la función `aplicarVelocidad3D(double m)` . En un modelo en minutos `second()` vale 1/60, así que la escala queda en m segundos de carrera por segundo real:

```
velocidad3D = m;
getEngine().setRealTimeMode( true );
getEngine().setRealTimeScale( m * second() );
```

3. Al principio de `irA(seccion)` , guardar la escala al entrar a la vista 3D y devolverla al salir:

```
boolean entra3D = seccion.equals( "MAPA3D" ) && !"MAPA3D".equals( pantalla );
boolean sale3D = !seccion.equals( "MAPA3D" ) && "MAPA3D".equals( pantalla );
if ( entra3D ) {
        escalaFuera3D = getEngine().getRealTimeScale();
        tiempoRealFuera3D = getEngine().getRealTimeMode();
        aplicarVelocidad3D( velocidad3D );
} else if ( sale3D ) {
        getEngine().setRealTimeMode( tiempoRealFuera3D );
        getEngine().setRealTimeScale( escalaFuera3D );
}
```

4. En PuestoDetalle3D, debajo de las cámaras, agregar los botones Real, x2, x5, x15 y x60 (rectángulo redondeado + texto). *On click*: `main.aplicarVelocidad3D( 15 );`. *Fill color*: `Math.abs( velocidadActual() - 15 ) < 0.01 ? Main.COLOR_TAB_ACTIVA : Main.COLOR_TAB`, donde `velocidadActual()` devuelve `getEngine().getRealTimeScale() / second()`. Así el botón activo se pinta solo, aunque la velocidad se cambie desde otro lado.

Con "Real" activo, la barra de AnyLogic muestra x0.02 (1/60). Si en la vista 3D se usa la barra de AnyLogic, x1 vuelve a ser 60 veces más rápido que la realidad: conviene usar los botones del panel.

Modelos animados de personas. AnyLogic trae personas en formato .glb en la paleta *3D Objects → People* (en disco, en `plugins/com.anylogic.ui_*/resources/3d`). Al arrastrar una a la escena, AnyLogic copia el archivo a la carpeta `3d/` del modelo. Cada modelo trae materiales con nombre (`body`, `hair`, `tshirt`, `shorts`…) y un conjunto de poses.

Las poses son cuadros fijos, no animaciones: `run_left` es la pierna izquierda adelante, `run_right` la derecha, y lo mismo con `walk_left` y `walk_right`. Para que una persona corra o camine hay que alternarlas, con la propiedad dinámica *Animation pose* del 3D Object (en el XML, `SelectedAnimationPoseCode`), que AnyLogic evalúa en cada cuadro. El color de cada material se cambia en *Colors* de las propiedades (fijo) o con `figura.setColor( "material", color )` en código (distinto para cada persona).

Quién	Modelo	Poses que usa	Ropa
Corredores	ped_sport_1_f, ped_sport_1_m	run_left/run_right, walk_left/walk_right, idle	Remera del color de la categoría; piel, pelo, short y zapatillas al azar
Voluntarios y ayudantes de mesa	ped_3_f, ped_3_m	walk_left/walk_right, stand_desk, carry, idle	Remera celeste y pantalón oscuro fijos; piel y pelo al azar

Quién	Modelo	Poses que usa	Ropa
Público	ped_3_f, ped_3_m, ped_1_f, ped_5_m, ped_5_f, child_1_m, child_1_f	idle, stand; los chicos, pointing	Combinaciones de una paleta, fijas en el XML
Transeúntes	ped_3_f, ped_3_m, ped_1_f, ped_5_m	walk_left/walk_right, idle	Al azar al crearse

Corredores (agente peatonPuesto).

1. Reemplazar la figura única por dos 3D Objects en (0, 0): `figuraCorredora` (ped_sport_1_f, *Visible*: `mujer`) y `figuraCorredor` (ped_sport_1_m, *Visible*: `!mujer`). En las dos, *Animation pose* = `pose()`.
2. Agregar las variables `mujer` (boolean) y `fasePaso` (double). Se asignan en `asignarCorredor` de PuestoDetalle3D: `p.mujer = randomTrue( 0.3 )` (supuesto: 30 % de mujeres), `p.fasePaso = uniform( 0, 2 )` para que no pisen todos a la vez, y `p.vestir()`.
3. Crear `pose()`. Un paso dura 60 / cadencia segundos, y la cadencia es la de cada corredor (sección 2):

```
double v = getSpeed();          // m/s
if ( v < 0.3 ) return "idle";   // parado, esperando la botella
if ( v < 2.0 ) {                // frenando: camina
        int n = (int) Math.floor( time() / second() / 0.3 + fasePaso );
        return n % 2 == 0 ? "walk_left" : "walk_right";
}
double paso = corredor != null ? 60.0 / corredor.cadencia : 0.34;
int n = (int) Math.floor( time() / second() / paso + fasePaso );
return n % 2 == 0 ? "run_left" : "run_right";
```

4. Crear `vestir()`: a la figura visible le pone `setColor( "shirt", corredor.colorCategoria() )`, y piel, pelo, short y zapatillas de paletas guardadas en Main (`Main.PIELES`, `Main.PELOS`, `Main.SHORTS`, `Main.ZAPATILLAS`: variables `Color[]` estáticas y constantes), elegidas con `uniform_discr( 0, largo - 1 )`.

AnyLogic tiene `selectWalkAnimationPose(agente, intervalo, poseQuieto, poses...)`, que alterna poses con un intervalo fijo. Acá se escribió `pose()` para atar el ritmo a la cadencia de cada corredor.

Voluntarios. Las figuras pasan a ped_3_f y ped_3_m, con remera celeste fija en *Colors*. Su `pose()` usa `isMoving()`, porque se mueven con `moveTo` y su `getSpeed()` no baja a 0 cuando se quedan quietos. Caminando alterna `walk_left`, `idle`, `walk_right`, `idle`

cada 0,27 s; en la fase `TOMANDO` devuelve `stand_desk` (inclinado sobre la mesa); esperando con botella, `carry` (brazos hacia adelante, con la botella en las manos). Los dos ayudantes de atrás de la mesa son 3D Objects fijos con pose `stand_desk`.

Público. Las 33 figuras fijas en fila, que alternaban dos modelos, se reemplazaron por 82 personas en grupos de 1 a 4. Las ubica un script con semilla fija, sobre las dos veredas y fuera de la zona de voluntarios. Cada una mira a la calle, girada de 10° a 45° hacia los corredores que llegan, y viste colores de la paleta. Su *Animation pose* es `poseEspectador( k, chico )`: cada 5 segundos, con un desfase propio, pueden cambiar de postura (`idle`, `stand`), y los chicos señalan (`pointing`).

Transeúntes (agente nuevo).

1. Crear el tipo de agente `Transeunte` (*New → Agent type*, espacio continuo, velocidad 1,3 m/s) con cuatro 3D Objects (ped_3_f, ped_3_m, ped_1_f, ped_5_m), visibles según `modelo == k` y con *Animation pose* `pose()`.
2. En PuestoDetalle3D, crear una población `transeuntes` de 8 agentes.
3. *On startup*: `iniciar()` sortea modelo y ropa y llama a `nuevoRecorrido( true )`.
4. `nuevoRecorrido(enCualquierLugar)` elige sentido y carril en la vereda norte, y llama a `setSpeed( uniform( 1.1, 1.5 ), MPS )`, `jumpTo( x, y, 1.5 )` y `moveTo( destino, y, 1.5 )`. *On arrival*: `nuevoRecorrido( false )`, que lo hace arrancar de nuevo desde una punta.

Los siete modelos nuevos suman unos 27 MB a la carpeta `3d/` del repositorio. Si se agregan editando el XML en lugar de arrastrarlos desde la paleta, además hay que registrarlos en `_alp/ModelResources.xml`.

12. Orden del editor: cada elemento en su lugar

Después de tantos cambios, el lienzo de Main tenía 44 variables y parámetros y 29 funciones en tres columnas, sin orden y con algunos elementos encimados. En PuestoDetalle3D, las variables, funciones y el flujo de peatones estaban dibujados encima de la calle. El objetivo es que cada elemento esté en el agente que corresponde, agrupado por lo que afecta en la simulación y lejos de las pantallas.

Qué queda en Main. Los 21 parámetros quedan en Main: son los que aparecen en la pantalla del experimento (*Simulation*) y se cambian para cada escenario. Si pasaran a un agente, cada agente tendría su propio valor y ya no se podrían configurar desde ahí. También quedan las estadísticas de la carrera, la navegación entre pantallas y las funciones que llaman los bloques del diagrama de procesos.

Cambio	Antes	Ahora
Paletas de ropa, piel y pelo	`Main.PIELES`, `Main.ROPA`…	`PuestoDetalle3D.PIELES`…, la escena donde están las personas
`capacidadInicial(k)`	Función de Main	Función de Puesto, que lee los parámetros de Main
Totales de faltantes y no atendidos	Contadores de Main que se sumaban en paralelo a los de cada puesto	Se calculan al mostrarlos: `puesto.stream().mapToInt( p -> p.noAtendidosStock ).sum()`
`navigate`, `resumenCategorias`, `resumenPuestos`	Funciones de Main	Borradas: no las usaba nada desde el rediseño de Métricas

1. Para mudar una variable a otro agente: cortarla en Main (Ctrl+X), pegarla en el agente (Ctrl+V) y cambiar las referencias (`Main.PIELES` pasa a ser `PuestoDetalle3D.PIELES`). Al compilar, AnyLogic marca como error cada referencia que quedó vieja, y con doble clic en el error lleva al lugar exacto.
2. Para quitar un total repetido: borrar la variable de Main, borrar la línea que la incrementaba (por ejemplo `main.noAtendidosStock++` en Puesto) y, en el texto de Inicio o Métricas que la mostraba, poner la suma de los puestos.
3. Para agrupar: dibujar un rectángulo redondeado por categoría, sin relleno (*Fill color: No color*) para no tapar los íconos, con un texto de título y otro de detalle. Después, arrastrar adentro sus parámetros, variables, funciones y poblaciones.
4. Ubicar todo fuera de las View Areas. Al correr el modelo, las variables y parámetros también se dibujan en el lugar del lienzo donde están; si caen dentro de una View Area, aparecen en esa pantalla. En Main, los recuadros quedaron a la izquierda del mapa 2D (x negativo), donde no llega ninguna pantalla.

Recuadro de Main	Qué afecta	Elementos
Llegada de corredores	Cuántos corredores largan y de qué categoría	tasaLlegada, sortearCategoria
Clima e hidratación	Cuánto toman y cuándo deciden parar	temperaturaC, nivelCalor, consumoLiquidoMlHora, volumenVasoMl, gelesPorCorredor, frutaPorCorredor, ajustarTemperatura, decidirHidratacion

Recuadro de Main	Qué afecta	Elementos
Atención en los puestos	Voluntarios, cola y tiempo de atención	voluntariosPorPuesto, entregaEnMano, colaMaximaPorVoluntario, tiempoAtencion, hayDemasiadaCola, cambiarVoluntarios, poolDePuesto, seizeDePuesto
Stock y política de reposición	Capacidad de cada insumo y cuándo pedir más	capacidadAgua, capacidadIsotonica, capacidadFruta, capacidadGel, zNivelServicio, topePuntoReposicion
Logística: camión y zorra	Depósito, viaje del camión y descarga	latDeposito, lonDeposito, velocidadCamionKmh, factorRecorridoCalles, capacidadZorra, velocidadZorraKmh, despacharCamion, iniciarDescarga
Agentes de la simulación	Poblaciones y ubicación de los puestos en el mapa	puesto, puestosDetalle3D, camiones, operarios, puntoPuesto
Resultados de la carrera	Contadores y promedios de Inicio y Métricas	corredoresLargados, llegadosPorCategoria, sumaMinutosPorCategoria, sumaHorasPorKm, camionesDespachados, registrarLargada, registrarLlegada, horasPorKmPromedio, corredoresEnRuta, tiempoPromedioCategoria, ritmoPromedioCategoria, formatoHoras
Pantallas y navegación	Cambio de pantalla, velocidad de la vista 3D y tablas	pantalla, puesto3DActual, COLOR_TAB, COLOR_TAB_ACTIVA, velocidad3D, escalaFuera3D, tiempoRealFuera3D, irA, verPuesto3D, colorTab, aplicarVelocidad3D, celdaStock, colorCeldaStock, textoReposicionCorto

PuestoDetalle3D. En el lienzo, la calle con sus veredas, árboles y edificios ocupa de x = −1000 a x = 1200, así que la lógica pasó a la derecha (x ⩾ 1300), en cinco recuadros: flujo de peatones, corredores que entran a la escena, personas de la escena (con las paletas),

panel del puesto y ajustes, y cámaras. Al mover bloques de un diagrama hay que mover también sus conectores: en AnyLogic alcanza con seleccionar bloques y conectores juntos y arrastrarlos. En Voluntario solo se separaron dos funciones que habían quedado encimadas.

13. Librerías usadas y sus funciones

El modelo usa tres librerías de AnyLogic (Process Modeling, Pedestrian y GIS) más el motor básico de agentes y presentación. Abajo, cada función o bloque, para qué se usó y dónde.

Process Modeling Library: el flujo de cada corredor por los 15 puestos (pestaña Lógica).

Bloque / función	Para qué se usó	Dónde
`Source`	Genera corredores cada `exponential(tasaLlegada)` segundos. *On exit* sortea la categoría. `count()` = cuántos largaron	`sourceCorredores`
`MoveTo`	Mueve al corredor por la red GIS hasta el próximo puesto, con velocidad dinámica	`moveTo1..15`, `moveToLlegada`
`SelectOutput`	Decide si para o sigue (`decidirHidratacion`). *On exit true/false* lo manda a la escena 3D	`decisionHidratacion..14`
`Seize` / `Release`	Toma y libera un voluntario. `size()` = corredores esperando	`seizeVoluntariosN`, `releaseVoluntariosN`
`Delay`	Tiempo de atención. *On at exit* llama `Puesto.atender`	`delayHidratacionN`
`ResourcePool`	Voluntarios de cada puesto. `idle()` = libres; `set_capacity(n)` cambia la cantidad	`poolVoluntariosN`
`Sink`	Fin de la carrera. *On enter* registra el tiempo; `count()` = llegados	`sinkCorredores`

Pedestrian Library: los corredores y la entrega en mano en cada escena 3D.

Bloque / elemento	Para qué se usó
`PedSource`	`inject(1)` crea un peatón por cada corredor que pasa; *Comfortable speed* = la velocidad del corredor
`PedSelectOutput`	`condition1` separa a los que paran de los que siguen
`PedGoTo`	Lleva al peatón a un punto (el voluntario) o a una línea (la salida)
`PedWait`	Espera breve mientras recibe la botella; *On exit* llama `terminarEntrega`
`PedSink`	Saca al peatón de la escena y suelta la cámara que lo seguía
Target Line, *Rectangular Area*	Línea de llegada, línea de salida y área de la calle (140 m × 14 m)

GIS y movimiento de agentes: camiones y operarios sobre el mapa real.

Función	Para qué se usó
`jumpTo(lat, lon)` / `jumpTo(nodo)`	Ubicar el camión en el depósito y cada puesto en su `GISPoint`
`moveTo(lat, lon)`	Camión por calles reales (ruteo del mapa)
`moveToInTime(lat, lon, t, MINUTE)`	Camión que llega en exactamente t minutos
`moveToStraight(lat, lon)`	Operario con zorra caminando en línea recta
`timeToArrival(MINUTE)`	Minutos que le faltan al camión
On arrival to target location	Qué hace cada agente al llegar, según su `fase`

Motor de agentes: poblaciones, eventos y números aleatorios.

Función	Para qué se usó
`add_camiones()`, `remove_camiones(c)`	Crear y borrar camiones y operarios en la corrida

Función	Para qué se usó
`getIndex()`	Número de puesto o escena dentro de su población
`create_X(t, MINUTE)`	Eventos dinámicos: `MedirDemanda`, `FinCarga`, `FinDescarga`, `TomarBotella`
`uniform`, `normal(sigma, media)`, `randomTrue(p)`, `exponential`	Sorteos: categoría, cadencia, zancada, decisión de parar, llegadas
`time()`, `second()`	Tiempo de simulación (en minutos) y conversión de segundos
`getEngine().setRealTimeScale(m)`, `getRealTimeScale()`, `setRealTimeMode(b)`	Velocidad de la simulación en unidades de modelo por segundo real. La vista 3D usa m · second() para ir en tiempo real
`getSpeed()`, `isMoving()`	Saber si una persona corre, camina o está quieta, para elegir su pose
`setSpeed(v, MPS)`, `jumpTo(x, y, z)`, `moveTo(x, y, z)`	Recorrido de los transeúntes por la vereda
`uniform_discr(a, b)`	Elegir al azar un color de las paletas de ropa, piel y pelo
`poblacion.stream().mapToInt( p -> p.dato ).sum()`	Sumar un dato de todos los agentes de una población: totales de faltantes y no atendidos en Inicio y Métricas

Presentación y 3D: pantallas, controles y gráficos.

Elemento / función	Para qué se usó
View Area + `navigateTo()`	Cada pantalla; `irA()` cambia entre ellas
Figuras con *On click* y *Fill color* dinámico	Pestañas, tarjetas, chips de puesto e insumo
Text con texto y color dinámicos	Indicadores, tablas y panel de estado
Group con *Scale X* dinámico	Barras de stock del panel 3D
Slider + `setValue(v, false)`	Ajustes por puesto y temperatura
3D Window + `setCamera(cam, true, ms)`	Cambio suave entre las cuatro cámaras
Camera dentro de un agente	Cámara que sigue al corredor

Elemento / función	Para qué se usó
3D Object (*Auto scale*), materiales `tarmac`, `concrete`, `grass`, `water`, *Environment* `SKYBOX_1`	Calle, edificios, árboles, faroles y cielo
Time Plot	Stock en tiempo real por insumo y puesto
3D Object de persona (.glb) con Animation pose dinámica	Corredores, voluntarios, público y transeúntes animados, alternando poses fijas en cada cuadro
`Shape3DObject.setColor(material, color)`	Ropa, piel y pelo distintos para cada persona; remera del color de la categoría

Fuera de AnyLogic se usaron el shapefile del recorrido (`.dbf`) y OpenStreetMap (Overpass y Nominatim) para los datos reales de cada puesto. Los dos se procesaron una sola vez y quedaron escritos en `Puesto.configurar()`.

14. Problemas encontrados y cómo se resolvieron

Problema	Causa	Solución
La corrida abortaba en `pedSeHidrata` ("the choice changed before the agent transmission")	La condición se evalúa dos veces, y los datos del corredor se cargaban entre una y otra	Asignar los datos al crear el peatón, una sola vez (`asignarCorredor` idempotente)
Los 15 puestos compartían voluntarios	Todos los `seizeVoluntariosN` apuntaban a `poolVoluntarios`	Cada uno toma su `poolVoluntariosN`
El puesto 1 se quedaba sin agua	El pedido salía con 280 vasos, que duraban menos que el viaje del camión	Punto de reposición por demanda durante la entrega (sección 6) y capacidades a escala real
La población de puestos no tenía datos en un clon limpio	Se cargaba de una base de datos que no estaba en el repositorio	15 agentes fijos y datos en `configurar()`
El gráfico mostraba stock negativo	Escala automática con todas las series en 0	Eje fijo de 0 a la capacidad

Problema	Causa	Solución
Los clics sobre figuras no funcionaban	Dos visores conectados a la misma corrida	Un solo visor: la ventana de AnyLogic
Las cámaras miraban para otro lado	Se supuso que Rotation Z = 0 era el norte	Rotation Z = 0 es el este; positivo gira hacia el sur
"Unresolved reference: airport"	Referencia a una librería que no usa ningún elemento	Se quitó del `.alpx`
Los corredores cruzaban la escena 3D en 2 segundos	El modelo mide en minutos: x1 es un minuto de carrera por segundo real	Escala m · second() al entrar a la vista 3D y la anterior al salir (sección 11)
Las personas se deslizaban sin mover las piernas	Las poses de los .glb son cuadros fijos, no animaciones	Alternarlas con Animation pose dinámica, al ritmo de la cadencia
No se podían agregar funciones a peatonPuesto editando el XML	Un agente sin funciones no tiene carpeta Code/ ni la declaración en su AOC	Crear Code/ y declarar <Functions> en el AOC (desde AnyLogic se hace solo)

Para probar cambios hechos fuera de AnyLogic, conviene cerrar el modelo desde el panel Projects (clic derecho → *Close*) y volver a abrirlo. Si AnyLogic avisa que hay un autosave más nuevo, hay que elegir *Discard autosave and open original model*.